به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس الگوریتم پیشرفته

تمرین چهارم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	810103098
تاریخ ارسال گزارش	1404/02/08
استاد	دکتر هشام فیلی
ترم	بهار 1404

فهرست

سوال 1 – رستم، سیاوش و گذرگاه های کوهستانی 5

1.1 صورت سوال: 5

1.2 پاسخ سوال بخش الف: 6

1.3 پاسخ سوال بخش ب: 7

1.4 پاسخ سوال بخش ج: 9

سوال 2 – ماموریت نهایی – مسیرهای فرار 10

2.1 پاسخ سوال: 11

سوال 3 – فرصت کارآموزی 14

3.1 پاسخ سوال: 14

3.2 الگوریتم پیشنهادی: تطبیق دوبخشی با حداکثر جریان 15

3.3 تحلیل پیچیدگی زمانی 17

3.4 مثال: 17

سوال 4 – ایزوله سازی ذهن آلوده 19

4.1 پاسخ سوال: 20

4.2 الگوریتم پیشنهادی 20

4.3 مثال 21

4.4 تحلیل الگوریتم 21

سوال5 – دورهمی پژوهشگران 22

5.1 پاسخ سوال: 23

5.2 الگوریتم پیشنهادی 23

5.3 تحلیل پیچیدگی زمانی 25

5.4 مثال 26

سوال6 – دورهمی پژوهشگران 27

6.1 پاسخ سوال: 28

6.2 الگوریتم پیشنهادی 28

6.3 تحلیل پیچیدگی زمانی..... 29
6.4 مثال..... 29

شکل‌ها

No table of figures entries found.

جدول‌ها

No table of figures entries found.

سوال 1 – رستم، سیاوش و گذرگاه های کوهستانی

1.1 صورت سوال:

سوال ۱: رستم، سیاوش و گذرگاه‌های کوهستانی

در روزگاری که مرزهای ایران زمین در آرامش بودند، ناگهان خبر آمد که افراسیاب به همراه برادرش گرسیوَرز، پیمانی دیرین را شکسته و با سپاهی عظیم، به مرزهای ایران تاخته‌اند. این خبر به کاخ شاه کاووس رسید و شاه با خشم فرمان داد: «سیاوش، فرزند برومندم، با دلاور ایران، رستم دستان، راهی شوید و پاسداران ایران‌زمین را به یاری گیرید.» سیاوش و رستم، سپاهیان خود را از سیستان (رأس مبدأ s) به سمت مرزهای توران (رأس مقصد t) حرکت می‌دهند. اما تنها راه رسیدن به میدان نبرد، عبور از گذرگاه‌های سخت‌گذر کوهستانی است؛ هر گذرگاه (یال گراف) ظرفیت مشخصی برای عبور سپاه دارد — برخی باریک، برخی محافظت‌شده، و برخی دیگر دره‌هایی خطرناک‌اند. اکنون، پس از بررسی و محاسبه، بیشینه‌ی تعداد نیروهایی که می‌توانند با موفقیت از سیستان به میدان نبرد برسند محاسبه شده است این مقدار در قالب جریان بیشینه f بر شبکه جریان $F = (G, c)$ مشخص است؛ که در آن $G = (V, E)$ گراف مسیرهاست، و c تابع ظرفیت یال‌هاست. اما ناگاه، خبر تازه‌ای می‌رسد: یکی از گذرگاه‌ها بین دو قله u و v که نقش مهمی در مسیر نیروها دارد، دچار تغییر شده — یا در اثر ریزش کوه بخشی از آن تخریب شده، یا به یاری مردم آن دیار، ترمیم و تقویت گشته است. ظرفیت جدید این گذرگاه، r است که با مقدار قبلی c(u, v) فرق دارد.

الف) ثابت کنید که:

۱. افزایش ظرفیت یک یال به اندازه ۱، حداکثر ۱ واحد به جریان بیشینه اضافه می‌کند.

۲. افزایش ظرفیت یک یال به اندازه k، حداکثر k واحد به جریان بیشینه اضافه می‌کند.

۳. کاهش ظرفیت یک یال به اندازه ۱، حداکثر ۱ واحد از جریان بیشینه کم می‌کند.

۴. کاهش ظرفیت یک یال به اندازه k، حداکثر k واحد جریان بیشینه را کاهش می‌دهد.

ب) خبر می‌رسد که گذرگاه میان دو قله‌ی u و v که پیش‌تر با ظرفیت $c(u,v)$ امکان عبور نیروها را فراهم می‌کرد، به یاری مردم آن سرزمین بازسازی شده و اکنون با ظرفیتی جدید برابر با r در اختیار سپاه ایران است، به‌گونه‌ای که $r > c(u,v)$. رستم که پیش‌تر با تدبیر و هماهنگی، بیشینه‌ی جریان نیروها را از سیستان به مرز توران محاسبه کرده بود، اکنون می‌خواهد بداند که آیا با این افزایش ظرفیت، امکان عبور نیروهای بیشتری وجود دارد یا نه، و اگر بله، چگونه می‌توان از این فرصت بهره برد.

الگوریتمی طراحی کنید که با استفاده از جریان بیشینه‌ی فعلی f و با در نظر گرفتن ظرفیت جدید $\acute{c}(u,v) = r$، بیشینه‌ی جدید جریان را در گراف تغییر یافته $\acute{F} = (G, \acute{c})$ محاسبه کند. الگوریتم شما باید تنها با تغییر همین یال، و بدون محاسبه‌ی مجدد کل جریان از ابتدا، در زمانی برابر با $O(k.(V+E))$ اجرا شود، که در آن $k = |c(u,v) - r|$ است.

صحت الگوریتم خود را اثبات کرده و نشان دهید چگونه می‌توان با حداکثر k مرحله افزایش جریان، به پاسخ درست رسید.

ج) اما اگر آن گذرگاه حیاتی دچار حادثه‌ای شده باشد و اکنون ظرفیت آن کاهش یابد، یعنی $r < c(u,v)$ رستم باید با دقت و تدبیر، جریان نیروها را متناسب با وضعیت جدید بازتنظیم کند تا از بروز گره در مسیر عبور سپاه جلوگیری شود.

الگوریتمی طراحی کنید که اگر مقدار جریان جاری روی یال (u,v) بیشتر از ظرفیت جدید r باشد، جریان را به صورت صحیح و پایدار کاهش دهد و سپس، اگر ممکن بود، جریان را در مسیرهای دیگر افزایش داده تا به بیشینه‌ی جدید برسد. الگوریتم باید از جریان فعلی f استفاده کرده و تنها با اصلاح لازم در جریان و حداکثر k مرحله افزایش احتمالی جریان، در زمانی برابر با $O(k.(V+E))$ اجرا شود.

درستی الگوریتم، حفظ شرایط تعادل جریان، و تحلیل دقیق زمانی آن را بیان نمایید.

1.2 پاسخ سوال بخش الف:

بخش الف: اثبات تأثیر تغییر ظرفیت یال بر جریان بیشینه

1. افزایش ظرفیت یک یال به اندازه 1، حداکثر 1 واحد به جریان بیشینه اضافه می‌کند.

- این گزاره صحیح است. وقتی ظرفیت یک یال (v,u) به اندازه 1 افزایش می‌یابد، در بهترین حالت این یال در مسیر افزایشی قرار می‌گیرد که می‌تواند حداکثر 1 واحد به جریان بیشینه اضافه کند.

اثبات :فرض کنید f جریان بیشینه در شبکه اصلی باشد و f جریان بیشینه در شبکه با ظرفیت افزایش‌یافته. طبق قضیه max-flow min-cut ، مقدار جریان بیشینه برابر با ظرفیت کمترین برش

است. اگر یال (u,v) در برش حداقل قرار نداشته باشد، افزایش ظرفیت آن تأثیری بر جریان بیشینه ندارد. اگر در برش حداقل باشد، افزایش ظرفیت آن به اندازه 1 واحد، ظرفیت برش را حداکثر 1 واحد افزایش می‌دهد. بنابراین $|f'|-|f| \leq 1$.

2. افزایش ظرفیت یک یال به اندازه k، حداکثر k واحد به جریان بیشینه اضافه می‌کند.

این گزاره نیز صحیح است. با استدلال مشابه، افزایش ظرفیت به اندازه k واحد می‌تواند حداکثر k واحد به جریان بیشینه اضافه کند.

اثبات :با استدلال مشابه، اگر یال (u,v) در برش حداقل باشد، افزایش ظرفیت آن به اندازه kk واحد، ظرفیت برش را حداکثر kkواحد افزایش می‌دهد. بنابراین $|f'|-|f| \leq k$.

3. کاهش ظرفیت یک یال به اندازه 1، حداکثر 1 واحد از جریان بیشینه کم می‌کند:

این گزاره صحیح است. اگر یالی که ظرفیتش کاهش می‌یابد در مسیر جریان بیشینه قرار داشته باشد، حداکثر به اندازه کاهش ظرفیت (1 واحد) از جریان بیشینه کم می‌شود.

اثبات :فرض کنید f جریان بیشینه در شبکه اصلی و f' جریان بیشینه در شبکه با ظرفیت کاهش‌یافته باشد. اگر یال (u,v) در هیچ برش حداقل قرار نداشته باشد یا جریان عبوری از آن کمتر از ظرفیت جدید باشد، کاهش ظرفیت تأثیری بر جریان بیشینه ندارد. در بدترین حالت، اگر یال در برش حداقل باشد و کاملاً اشباع باشد، کاهش ظرفیت به اندازه 1 واحد، جریان بیشینه را حداکثر 1 واحد کاهش می‌دهد. بنابراین $|f|-|f'| \leq 1$.

4. کاهش ظرفیت یک یال به اندازه k، حداکثر k واحد از جریان بیشینه را کاهش می‌دهد.

این گزاره صحیح است. با استدلال مشابه، کاهش ظرفیت به اندازه k واحد، حداکثر k واحد از جریان بیشینه را کاهش می‌دهد.

اثبات :با استدلال مشابه، کاهش ظرفیت یال (u,v) به اندازه k واحد، در بدترین حالت جریان بیشینه را حداکثر kkواحد کاهش می‌دهد. بنابراین $|f|-|f'| \leq k$.

1.3 پاسخ سوال بخش ب:

برای این بخش، از الگوریتم فورد-فولکرسون استفاده می‌کنیم که یکی از الگوریتم‌های استاندارد برای محاسبه جریان بیشینه است:

الگوریتم محاسبه جریان بیشینه جدید با افزایش ظرفیت:

ورودی: گراف G(V,E)، جریان بیشینه فعلی f، ظرفیت جدید r برای یال (u,v) که $r > c(u,v)$

خروجی: جریان بیشینه جدید F

1. ظرفیت یال (u,v) را از c(u,v) به r افزایش دهید.
2. شبکه باقیمانده Gf را با توجه به جریان فعلی f ایجاد کنید:

a. برای هر یال (i,j) با جریان f(i,j) و ظرفیت c(i,j):

i. اگر f(i,j) < c(i,j)، یک یال رو به جلو (i,j) با ظرفیت c(i,j) - f(i,j) در Gf قرار دهید.

ii. اگر f(i,j) > 0، یک یال رو به عقب (j,i) با ظرفیت f(i,j) در Gf قرار دهید.

b. برای یال (u,v) که ظرفیتش افزایش یافته:

i. ظرفیت یال رو به جلو (u,v) در Gf را به c(u,v) - f(u,v) + (r - c(u,v)) = r - f(u,v) افزایش دهید.

3. با استفاده از الگوریتم فورد-فولکرسون، مسیرهای افزایشی در Gf را پیدا کنید:

a. تا زمانی که مسیر افزایشی از s به t در Gf وجود دارد:

i. با استفاده از جستجوی اول عمق (DFS) یا جستجوی اول سطح (BFS)، یک مسیر افزایشی p پیدا کنید.

ii. ظرفیت باقیمانده کمینه در مسیر p را به عنوان cf(p) محاسبه کنید.

iii. جریان را در مسیر p به اندازه cf(p) افزایش دهید:

- برای هر یال رو به جلو (i,j) در p، f(i,j) را cf(p) واحد افزایش دهید.

- برای هر یال رو به عقب (i,j) در p، f(j,i) را cf(p) واحد کاهش دهید.

iv. شبکه باقیمانده Gf را بروزرسانی کنید.

4. جریان نهایی f را به عنوان F برگردانید.

1- **صحت الگوریتم** :این الگوریتم با استفاده از مسیرهای افزایشی در شبکه باقیمانده، جریان را تا حد امکان افزایش می‌دهد. چون ما فقط ظرفیت یک یال را افزایش داده‌ایم، نیازی به محاسبه مجدد کل جریان نیست و می‌توانیم از جریان فعلی شروع کنیم.

2- **تحلیل زمانی** :در بدترین حالت، ممکن است نیاز به k = r - c(u,v) مرحله افزایش جریان داشته باشیم (هر بار 1 واحد). هر مرحله جستجو در گراف O(V+E) زمان می‌برد، بنابراین پیچیدگی کلی O(k(V+E)) است.

3- این الگوریتم با پیچیدگی زمانی O(k(V+E)) اجرا می‌شود، که k = r - c(u,v) است (حداکثر تعداد مسیرهای افزایشی که می‌توان پیدا کرد).

4- اثبات درستی:

چون ما از جریان معتبر فعلی شروع می‌کنیم، شرایط تعادل جریان از قبل برقرار است.

با افزایش ظرفیت یال (u,v)، حداکثر می‌توانیم k = r - c(u,v) واحد جریان اضافه کنیم.

الگوریتم Ford-Fulkerson تضمین می‌کند که اگر مسیر افزایشی وجود داشته باشد، آن را پیدا می‌کند.

در نهایت، وقتی دیگر مسیر افزایشی وجود نداشته باشد، به جریان بیشینه جدید رسیده‌ایم.

1.4 پاسخ سوال بخش ج:

بخش ج: الگوریتم برای کاهش جریان در صورت حادثه

الگوریتم تنظیم جریان پس از کاهش ظرفیت:

ورودی: گراف G(V,E)، جریان بیشینه فعلی f، ظرفیت جدید r برای یال (u,v) که r < c(u,v)

خروجی: جریان بیشینه جدید F

1. اگر f(u,v) ≤ r:
   a. ظرفیت یال (u,v) را به r کاهش دهید.
   b. جریان فعلی f همچنان معتبر است.
   c. F = f را برگردانید.

2. اگر f(u,v) > r:
   a. مقدار excess = f(u,v) - r را محاسبه کنید (مقدار جریان اضافی).
   b. جریان در یال (u,v) را به r کاهش دهید: f(u,v) = r.
   c. برای حفظ شرایط تعادل جریان، جریان excess را به عقب برگردانید:
      i. جریان از v به u به اندازه excess واحد برگردانید (این کار با ایجاد جریان منفی در یال (u,v) یا مثبت در یال (v,u) انجام می‌شود).
   d. شبکه باقیمانده Gf را با توجه به جریان اصلاح‌شده ایجاد کنید.
   e. یک شبکه کمکی G' ایجاد کنید:
      i. گره‌های G' همان گره‌های G هستند.
      ii. یال‌های G' همان یال‌های شبکه باقیمانده Gf هستند.
   f. با استفاده از الگوریتم فورد-فولکرسون، سعی کنید excess واحد جریان را از v به t هدایت کنید:
      i. تا زمانی که مسیر افزایشی از v به t در G' وجود دارد و هنوز excess > 0:
         - یک مسیر افزایشی p پیدا کنید.
         - ظرفیت باقیمانده کمینه در مسیر p را به عنوان cf(p) محاسبه کنید.
         - مقدار flow = min(cf(p), excess) را محاسبه کنید.
         - جریان را در مسیر p به اندازه flow افزایش دهید.
         - excess را به اندازه flow کاهش دهید.
         - شبکه G' را بروزرسانی کنید.

g. اگر هنوز excess > 0، سعی کنید excess واحد جریان را از s به u هدایت کنید:

i. تا زمانی که مسیر افزایشی از s به u در 'G وجود دارد و هنوز excess > 0:

- یک مسیر افزایشی p پیدا کنید.
- ظرفیت باقیمانده کمینه در مسیر p را به عنوان cf(p) محاسبه کنید.
- مقدار flow = min(cf(p), excess) را محاسبه کنید.
- جریان را در مسیر p به اندازه flow افزایش دهید.
- excess را به اندازه flow کاهش دهید.
- شبکه 'G را بروزرسانی کنید.

h. اگر هنوز excess > 0، جریان بیشینه به اندازه excess کاهش می‌یابد.

i. جریان نهایی را به عنوان F برگردانید.

صحت الگوریتم :این الگوریتم ابتدا جریان را در یال آسیب‌دیده به میزان مجاز کاهش می‌دهد. سپس با استفاده از مسیرهای افزایشی در شبکه باقیمانده، سعی می‌کند جریان کاهش‌یافته را از طریق مسیرهای دیگر جبران کند. اگر مسیر کافی برای جبران کل جریان کاهش‌یافته وجود نداشته باشد، جریان بیشینه کاهش می‌یابد.

حفظ شرایط تعادل جریان :در هر مرحله از الگوریتم، شرایط تعادل جریان در همه گره‌ها حفظ می‌شود. هنگامی که جریان در یال (u,v) کاهش می‌یابد، برای حفظ تعادل، جریان excessبه عقب برگردانده می‌شود و سپس از طریق مسیرهای دیگر هدایت می‌شود.

تحلیل زمانی :در بدترین حالت، ممکن است نیاز به k = f(u,v) - r مرحله افزایش جریان داشته باشیم. هر مرحله جستجو در گراف O(V+E) زمان می‌برد، بنابراین پیچیدگی کلی O(k(V+E))است.

سوال 2 – ماموریت نهایی – مسیرهای فرار

در یک عملیات بی‌سابقه، پروفسور با تیمی از متخصصان نفوذ، نقشه‌ای برای دستبرد هم‌زمان به چند خزانه‌ی محرمانه طراحی کرده است. هر تیم، از یک نقطه‌ی متفاوت در شهر به نقشه وارد می‌شود. اما همه‌ی آن‌ها باید پس از انجام مأموریت، در نقطه‌ای مشخص و واحد (نقطه‌ی فرار) به هم برسند؛ جایی که وسیله‌ی خروج نهایی منتظر آن‌هاست — یک هلیکوپتر، که زمان دقیق حرکتش از پیش تعیین شده است. اما شرط حیاتی عملیات چنین است: به‌هیچ‌وجه نباید هیچ‌یک از تیم‌ها در مسیر خروج‌شان، از یک مسیر مشترک با تیم دیگر عبور کنند. آن‌ها می‌توانند از برخی نقاط مشترک عبور کنند، اما مسیرها (یعنی راه‌ها، تونل‌ها یا گذرگاه‌ها) باید به‌طور کامل از هم جدا باشند. تداخل در حتی یک مسیر، به‌معنای شناسایی توسط سیستم امنیتی است و باعث شکست کامل عملیات خواهد شد.

اکنون نقشه‌ی کامل شهر به‌صورت مجموعه‌ای از مسیرهای یک‌طرفه در اختیار شماست که مبدأهای تیم‌ها را به نقطه‌ی فرار متصل می‌کنند. پروفسور از شما کمک می‌خواهد تا الگوریتمی ارائه دهید برای بررسی اینکه آیا می‌توان برای هر تیم، یک مسیر فرار اختصاصی و بدون اشتراک طراحی کرد یا نه. اگر ممکن است، آن مسیرها را الگوریتم شما بیابد؛ در غیر این‌صورت، تنها گزینه‌ی امن، لغو کامل عملیات است. در نهایت، پیچیدگی زمانی الگوریتم پیشنهادی خود را نیز تحلیل و گزارش کنید.

2.1 پاسخ سوال:

با توجه به صورت سؤال، ما باید الگوریتمی طراحی کنیم که برای هر تیم، یک مسیر فرار اختصاصی بدون اشتراک با مسیرهای دیگر تیم‌ها تعیین کند. این مسئله به یک مسئله جریان چندگانه با مسیرهای مجزا (Edge-Disjoint Paths) مرتبط است که در منابع استاندارد الگوریتم مانندCLRS ، داسگوپتا و کلاینبرگ به آن پرداخته شده است.

بررسی مسئله:

1. ورودی:

نقشه شهر به صورت گراف G(V,E)

مجموعه‌ای از نقاط شروع $s_1, s_2, \ldots, s_k$ (محل ورود تیم‌ها)

یک نقطه پایان مشترک t (نقطه فرار)

2. هدف:

یافتن k مسیر مجزا (بدون اشتراک یال) از هر نقطه شروع s_i به نقطه پایان t

3. شرط:

مسیرها باید کاملاً از هم جدا باشند (هیچ یالی نباید بین دو مسیر مشترک باشد)

الگوریتم پیشنهادی:

برای حل این مسئله از الگوریتم جریان بیشینه و قضیه مینکات استفاده می‌کنیم:

الگوریتم یافتن مسیرهای فرار مجزا:

ورودی: گراف $G(V,E)$، نقاط شروع $s_1, s_2, ..., s_k$، نقطه پایان t

خروجی: k مسیر مجزا یا اعلام عدم امکان

1. یک گراف جدید $G'(V',E')$ ایجاد کنید:
   a. تمام گره‌ها و یال‌های G را در G' کپی کنید
   b. یک گره منبع مجازی s به G' اضافه کنید
   c. برای هر نقطه شروع s_i، یک یال (s,s_i) با ظرفیت 1 به G' اضافه کنید
   d. برای تمام یال‌های G'، ظرفیت 1 قرار دهید

2. با استفاده از الگوریتم فورد-فولکرسون، جریان بیشینه از s به t در G' را محاسبه کنید

3. اگر مقدار جریان بیشینه برابر با k باشد:
   a. مسیرهای مجزا وجود دارند
   b. با استفاده از جریان محاسبه‌شده، k مسیر مجزا را بازسازی کنید:
      i. برای هر نقطه شروع s_i، یال‌هایی که در آنها جریان 1 وجود دارد را دنبال کنید
      ii. این یال‌ها مسیرهای مجزا از s_i به t را تشکیل می‌دهند

4. اگر مقدار جریان بیشینه کمتر از k باشد:
   a. امکان یافتن k مسیر مجزا وجود ندارد
   b. "عملیات با شکست مواجه شد" را برگردانید

توضیح مراحل الگوریتم:

مرحله 1: ساخت گراف کمکی

ابتدا یک گراف کمکی می‌سازیم که در آن یک گره منبع مجازی s به تمام نقاط شروع متصل می‌شود. این تبدیل، مسئله یافتن k مسیر مجزا از نقاط مختلف به یک مقصد را به مسئله یافتن k مسیر مجزا از یک منبع به یک مقصد تبدیل می‌کند.

مرحله 2: محاسبه جریان بیشینه

با استفاده از الگوریتم فورد-فولکرسون، جریان بیشینه از s به t را محاسبه می‌کنیم. از آنجا که ظرفیت همه یال‌ها 1 است، هر مسیر افزایشی دقیقاً 1 واحد به جریان اضافه می‌کند.

مرحله 3: بازسازی مسیرها

اگر مقدار جریان بیشینه برابر با k باشد، می‌توانیم k مسیر مجزا پیدا کنیم. برای بازسازی این مسیرها، از هر نقطه شروع s_i شروع می‌کنیم و یال‌هایی که در آنها جریان 1 وجود دارد را دنبال می‌کنیم تا به t برسیم.

مرحله 4: بررسی امکان‌پذیری

اگر مقدار جریان بیشینه کمتر از k باشد، نمی‌توان k مسیر مجزا پیدا کرد و عملیات با شکست مواجه می‌شود.

درستی الگوریتم:

درستی این الگوریتم بر اساس قضیه مینکات و قضیه مسیرهای مجزا استوار است:

1. **قضیه مسیرهای مجزا**: در یک گراف با ظرفیت 1 برای همه یال‌ها، تعداد مسیرهای مجزا از s به t برابر با مقدار جریان بیشینه از s به t است.
2. با ساختن گراف کمکی و اضافه کردن یک منبع مجازی s، مسئله یافتن k مسیر مجزا از نقاط مختلف به مسئله یافتن جریان بیشینه با مقدار حداقل k تبدیل می‌شود.
3. اگر جریان بیشینه برابر با k باشد، k مسیر مجزا وجود دارد و می‌توان آنها را از جریان محاسبه‌شده بازسازی کرد.

تحلیل پیچیدگی زمانی:

- ساخت گراف کمکی: $O(V+E)$
- الگوریتم فورد-فولکرسون: $O(k \cdot E)$ (زیرا حداکثر k مسیر افزایشی وجود دارد و هر مسیر با BFS در $O(E)$ زمان پیدا می‌شود)
- بازسازی مسیرها: $O(k \cdot E)$

بنابراین، پیچیدگی زمانی کل الگوریتم $O(k \cdot E)$ است، که در آن k تعداد تیم‌ها و E تعداد یال‌های گراف است.

بهینه‌سازی:

برای بهبود کارایی الگوریتم، می‌توان از نسخه بهبودیافته الگوریتم فورد-فولکرسون مانند الگوریتم ادموندز-کارپ استفاده کرد که مسیرهای افزایشی را با BFS پیدا می‌کند و پیچیدگی زمانی $O(V \cdot E^2)$ دارد.

همچنین، می‌توان از الگوریتم دینیک با پیچیدگی $O(V^2 \cdot E)$ یا الگوریتم پوش-ریلبل با پیچیدگی $O(V^2 \cdot \sqrt{E})$ استفاده کرد که برای گراف‌های بزرگ کارآمدتر هستند.

سوال 3 – فرصت کارآموزی

سوال ۳: فرصت کارآموزی

در یک برنامه‌ی بزرگ کارآموزی تابستانی، چندین شرکت معتبر بین‌المللی تعداد محدودی موقعیت شغلی برای دانشجویان ارائه کرده‌اند. این موقعیت‌ها شامل حوزه‌های متنوعی از جمله تحلیل داده، طراحی محصول، یادگیری ماشین، امنیت سایبری و... هستند. با توجه به محدودیت منابع، هر موقعیت تنها می‌تواند تعداد مشخصی از دانشجویان را بپذیرد، و این موضوع باعث می‌شود برخی از موقعیت‌های محبوب زود پر شوند و برخی از دانشجویان بدون جایگاه بمانند. برای افزایش رضایت، سامانه‌ی ثبت‌نام دانشگاه از دانشجویان می‌خواهد پیش از زمان نهایی انتخاب، فهرستی از موقعیت‌های مورد علاقه خود را ارسال کنند. سپس یک الگوریتم مرکزی، تخصیص موقعیت‌ها را بین دانشجویان انجام می‌دهد. دانشجویانی که هیچ‌یک از انتخاب‌هایشان را دریافت نکنند، به فهرست ذخیره (یا «کوپن فرصت بعدی») منتقل خواهند شد. هدف سیستم، کمینه کردن تعداد دانشجویانی است که در این فهرست قرار می‌گیرند.

فرض کنید در یک بازه‌ی مشخص، m موقعیت شغلی مختلف به نام‌های $b_1, b_2, \dots, b_m$ ارائه شده‌اند، و از هر موقعیت b_j تعداد q_j جایگاه موجود است. همچنین n دانشجو به نام‌های $a_1, \dots, a_n$ وجود دارند که هر a_i مجموعه‌ای از انتخاب‌های مورد علاقه خود را به‌صورت مجموعه‌ای به نام A_i ثبت کرده‌اند.

الگوریتمی ارائه دهید که برای هر دانشجو یکی از انتخاب‌های دلخواهش را تخصیص دهد، یا در صورت عدم امکان، او را به عنوان «رد شده» علامت‌گذاری کند. هدف نهایی، کمینه کردن تعداد دانشجویان رد شده است. در نهایت، پیچیدگی زمانی الگوریتم پیشنهادی خود را نیز تحلیل و گزارش کنید.

3.1 پاسخ سوال:

تحلیل مسئله

این مسئله یک مسئله تخصیص است که می‌توان آن را با استفاده از الگوریتم‌های جریان شبکه حل کرد. به طور خاص، این یک مسئله تطبیق دوبخشی (Bipartite Matching) است که در آن:

- m موقعیت شغلی $b_1, b_2, \dots, b_m$ داریم که هر موقعیت b_i دارای q_i جایگاه است.

- n دانشجو $a_1, a_2, \ldots, a_n$ داریم که هر دانشجو a_j فهرستی از موقعیت‌های مورد علاقه خود را ثبت کرده است.
- هدف، تخصیص دانشجویان به موقعیت‌های شغلی به گونه‌ای است که تعداد دانشجویان رد شده (دانشجویانی که هیچ موقعیتی به آنها تخصیص داده نشده) کمینه شود.

3.2 الگوریتم پیشنهادی: تطبیق دوبخشی با حداکثر جریان

برای حل این مسئله، می‌توانیم از الگوریتم حداکثر جریان در یک گراف دوبخشی استفاده کنیم:

الگوریتم تخصیص موقعیت‌های کارآموزی:

ورودی:

- مجموعه موقعیت‌های شغلی $B = \{b_1, b_2, \ldots, b_m\}$ با ظرفیت‌های $q_1, q_2, \ldots, q_m$

- مجموعه دانشجویان $A = \{a_1, a_2, \ldots, a_n\}$

- برای هر دانشجو a_j، مجموعه موقعیت‌های مورد علاقه $P_j \subseteq B$

خروجی: تخصیص بهینه دانشجویان به موقعیت‌ها که تعداد دانشجویان رد شده را کمینه می‌کند

1. یک گراف جهت‌دار $G(V,E)$ به شکل زیر بسازید:
   a. $V = \{s, t\} \cup A \cup B$، که s یک رأس منبع و t یک رأس چاه است
   b. برای هر دانشجو a_j، یک یال (s, a_j) با ظرفیت 1 اضافه کنید
   c. برای هر دانشجو a_j و هر موقعیت $b_i \in P_j$، یک یال (a_j, b_i) با ظرفیت 1 اضافه کنید
   d. برای هر موقعیت b_i، یک یال (b_i, t) با ظرفیت q_i اضافه کنید
2. الگوریتم حداکثر جریان (مانند Ford-Fulkerson یا Edmonds-Karp) را روی گراف G اجرا کنید
3. تخصیص نهایی را از جریان بیشینه استخراج کنید:
   a. برای هر دانشجو a_j و هر موقعیت b_i، اگر جریان در یال (a_j, b_i) برابر 1 باشد، دانشجو a_j را به موقعیت b_i تخصیص دهید
   b. برای هر دانشجو a_j، اگر هیچ یالی از a_j به هیچ موقعیتی جریان نداشته باشد، دانشجو a_j را به عنوان "رد شده" علامت‌گذاری کنید

4. تخصیص نهایی و تعداد دانشجویان رد شده را برگردانید

اثبات درستی الگوریتم

1. **ساختار گراف**: گراف G یک گراف دوبخشی است که دانشجویان را به موقعیت‌های شغلی متصل می‌کند. هر دانشجو حداکثر به یک موقعیت تخصیص داده می‌شود (ظرفیت یال‌های خروجی از s برابر 1 است) و هر موقعیت b_i حداکثر q_i دانشجو را می‌پذیرد (ظرفیت یال‌های ورودی به t برابر q_i است).
2. **بهینگی:** الگوریتم حداکثر جریان، بیشترین تعداد ممکن تخصیص را پیدا می‌کند. این به معنای کمینه کردن تعداد دانشجویان رد شده است.
3. **محدودیت های مسئله:** الگوریتم تضمین می‌کند که:

هر دانشجو حداکثر به یک موقعیت تخصیص داده می‌شود.

هر موقعیت b_i حداکثر q_i دانشجو را می‌پذیرد.

دانشجویان فقط به موقعیت‌های مورد علاقه خود تخصیص داده می‌شوند.

به عبارت دیگر و به زبان ساده تر

اثبات درستی الگوریتم

1. **مدل‌سازی صحیح:**

هر دانشجو حداکثر به یک موقعیت تخصیص می‌یابد (به دلیل ظرفیت 1 در یال‌های (s, a_j))

هر موقعیت b_i حداکثر q_i دانشجو می‌پذیرد (به دلیل ظرفیت q_i در یال‌های (b_i, t))

دانشجویان فقط به موقعیت‌های مورد علاقه خود تخصیص می‌یابند (به دلیل ساختار یال‌های (a_j, b_i))

2. **بهینگی:**

الگوریتم فورد-فولکرسون جریان بیشینه را محاسبه می‌کند

جریان بیشینه معادل حداکثر تعداد دانشجویانی است که می‌توانند به موقعیت‌های مورد علاقه خود تخصیص یابند

بنابراین، تعداد دانشجویان رد شده کمینه می‌شود

3.3 تحلیل پیچیدگی زمانی

1. **ساخت گراف:**

- تعداد رئوس: $|V| = 2 + n + m$ (منبع، چاه، دانشجویان و موقعیت‌ها)
- تعداد یال‌ها: $E \leq |n + nm + m|$ (در بدترین حالت، هر دانشجو به تمام موقعیت‌ها علاقه‌مند است)

2. **الگوریتم حداکثر جریان:**

- با استفاده از الگوریتم Edmonds-Karp: $O(|V|\cdot|E|^2) = O((n+m)\cdot(n+nm+m)^2)$
- در عمل، می‌توان از الگوریتم‌های کارآمدتر مانند Dinic با پیچیدگی $O(|V|^2\cdot|E|)$ یا Push-Relabel با پیچیدگی $O(|V|^2\cdot|E|^{(1/2)})$ استفاده کرد.

3. **استخراج تخصیص:** $O(|E|) = O(n + nm + m)$

4. **پیچیدگی کلی:**

- با استفاده از Edmonds-Karp: $O((n+m)\cdot(n+nm+m)^2)$
- با استفاده از Dinic: $O((n+m)^2\cdot(n+nm+m))$

تحلیل پیچیدگی زمانی

- ساخت گراف: $O(n + m + \Sigma_j|S_j|)$ که در آن $\Sigma_j|S_j|$ مجموع تعداد انتخاب‌های همه دانشجویان است
- الگوریتم فورد-فولکرسون با پیاده‌سازی ادموندز-کارپ: $O(VE^2) = O((n+m)^3)$
- استخراج تخصیص: $O(n + \Sigma_j|S_j|)$

پیچیدگی زمانی کل: $O((n+m)^3)$

3.4 مثال:

فرض کنید 3 موقعیت شغلی (b_1, b_2, b_3) با ظرفیت‌های $q_1=1, q_2=2, q_3=1$ و 4 دانشجو (a_1, a_2, a_3, a_4) داریم .انتخاب‌های دانشجویان:

- $S_1 = \{b_1, b_2\}$
- $S_2 = \{b_1, b_3\}$
- $S_3 = \{b_2\}$
- $S_4 = \{b_2, b_3\}$

گام 1 :ساخت گراف جریان شبکه:

- گره‌ها: $s, a_1, a_2, a_3, a_4, b_1, b_2, b_3, t$
- یال‌ها با ظرفیت‌ها:
  - $(s, a_1), (s, a_2), (s, a_3), (s, a_4)$ همگی با ظرفیت 1
  - $(a_1, b_1), (a_1, b_2), (a_2, b_1), (a_2, b_3), (a_3, b_2), (a_4, b_2), (a_4, b_3)$ همگی با ظرفیت 1
  - (b_1, t) با ظرفیت 1، (b_2, t) با ظرفیت 2، (b_3, t) با ظرفیت 1

گام 2 :محاسبه جریان بیشینه:

- مسیر افزایشی 1: $s \to a_1 \to b_1 \to t$ (جریان +1)
- مسیر افزایشی 2: $s \to a_3 \to b_2 \to t$ (جریان +1)
- مسیر افزایشی 3: $s \to a_4 \to b_2 \to t$ (جریان +1)
- مسیر افزایشی 4: $s \to a_2 \to b_3 \to t$ (جریان +1)

جریان بیشینه = 4 (تمام دانشجویان تخصیص یافته‌اند)

گام 3 :استخراج تخصیص:

- a_1 به b_1 تخصیص می‌یابد
- a_2 به b_3 تخصیص می‌یابد
- a_3 به b_2 تخصیص می‌یابد
- a_4 به b_2 تخصیص می‌یابد

تعداد دانشجویان رد شده = 0 (همه دانشجویان تخصیص یافته‌اند)

مثال دیگر (با دانشجوی رد شده)

فرض کنید 2 موقعیت شغلی (b_1, b_2) با ظرفیت‌های $q_1=1, q_2=1$ و 3 دانشجو (a_1, a_2, a_3) داریم.
انتخاب‌های دانشجویان:

- $S_1 = \{b_1\}$
- $S_2 = \{b_1, b_2\}$
- $S_3 = \{b_1\}$

گام 1 :ساخت گراف جریان شبکه

گام 2 :محاسبه جریان بیشینه:

- مسیر افزایشی 1: $s \to a_1 \to b_1 \to t$ (جریان +1)

- مسیر افزایشی 2: $s \to a_2 \to b_2 \to t$ (جریان +1)

جریان بیشینه = 2

گام 3 :استخراج تخصیص:

- a_1 به b_1 تخصیص می‌یابد
- a_2 به b_2 تخصیص می‌یابد
- a_3 رد می‌شود (چون نمی‌تواند به b_1 تخصیص یابد و انتخاب دیگری ندارد)

تعداد دانشجویان رد شده = 1

سوال 4 – ایزوله سازی ذهن آلوده

در یکی از مأموریت‌های نفوذ ذهنی، تیم Cobb در یک شبکه‌ی ذهنی چندلایه وارد شده‌اند تا ایده‌ای خاص را در ذهن هدف القا کنند. اما به‌طور ناگهانی متوجه می‌شوند که یک ذهن آلوده یا سمی در یکی از گره‌های ذهنی فعال شده و در حال انتشار افکار مخرب به سایر ذهن‌های پاک است.

۶

برای جلوگیری از نابودی مأموریت، تیم باید با استفاده از دیواره‌های ذهنی یا تغییر مسیرهای ارتباطی بین گره‌ها، ذهن آلوده را از دیگر ذهن‌ها ایزوله کند. مسیرهای ذهنی به‌صورت یک شبکه‌ی ارتباطی بدون‌جهت تعریف شده‌اند، که قطع هر مسیر هزینه‌ی مشخصی دارد. هدف این است که کمترین مجموع هزینه‌ی لازم برای قطع ارتباط ذهن آلوده با بقیه را به‌دست آورید. الگوریتمی طراحی کنید که این مقدار را در زمان چندجمله‌ای محاسبه کند. در نهایت، پیچیدگی زمانی الگوریتم پیشنهادی خود را نیز تحلیل و گزارش کنید.

4.1 پاسخ سوال:

تحلیل مسئله

در این مسئله با یک شبکه ذهنی چندجمله‌ای مواجه هستیم که یک ذهن آلوده در آن فعال شده و در حال انتشار افکار مخرب به سایر ذهن‌های پاک است. هدف ما طراحی الگوریتمی است که:

1. با تغییر مسیرهای ارتباطی بین گره‌ها، ذهن آلوده را از سایر ذهن‌ها ایزوله کند.
2. هزینه قطع هر مسیر ارتباطی مشخص است.
3. باید کمترین مجموع هزینه لازم برای قطع ارتباط ذهن آلوده با بقیه را محاسبه کنیم.

این مسئله در واقع یک مسئله برش کمینه (Minimum Cut) در نظریه گراف است.

4.2 الگوریتم پیشنهادی

برای حل این مسئله، می‌توانیم از الگوریتم برش کمینه-جریان بیشینه (Min-Cut Max-Flow) استفاده کنیم. طبق قضیه فورد-فالکرسون، مقدار برش کمینه بین دو گره s و t در یک گراف برابر با جریان بیشینه از s به t است.

الگوریتم ایزوله کردن ذهن آلوده:

ورودی: گراف G(V,E) با وزن‌های c(u,v) برای هر یال (u,v)∈E، گره ذهن آلوده s

خروجی: مجموعه یال‌هایی که باید قطع شوند و مجموع هزینه آنها

1. یک گره مجازی t به گراف اضافه کنید
2. تمام گره‌های ذهن پاک را به گره t با یال‌هایی با ظرفیت بی‌نهایت متصل کنید
3. الگوریتم برش کمینه-جریان بیشینه را بین گره s (ذهن آلوده) و گره t اجرا کنید:
   a. یک گراف جریان G' بسازید که ظرفیت هر یال (u,v) در آن برابر با c(u,v) باشد
   b. با استفاده از الگوریتم فورد-فالکرسون، جریان بیشینه از s به t را محاسبه کنید
   c. برش کمینه را با استفاده از جریان بیشینه به دست آورید:
      i. مجموعه S را با گره s شروع کنید
      ii. تمام گره‌هایی که از s قابل دسترس هستند (در شبکه باقیمانده) را به S اضافه کنید
      iii. برش کمینه شامل تمام یال‌های (u,v) است که u∈S و v∉S

4. مجموع هزینه برش کمینه و یال‌های آن را برگردانید

4.3 مثال

برای درک بهتر الگوریتم، یک مثال ساده را بررسی می‌کنیم:

فرض کنید شبکه ذهنی ما شامل 5 ذهن است: A (ذهن آلوده)، B، C، D و E (ذهن‌های پاک). یال‌ها و هزینه قطع آنها به شرح زیر است:

- (A,B): هزینه 3
- (A,C): هزینه 2
- (B,D): هزینه 1
- (C,D): هزینه 4
- (C,E): هزینه 2
- (D,E): هزینه 5

مراحل اجرای الگوریتم:

1. یک گره مجازی T اضافه می‌کنیم
2. گره‌های B، C، D و E را به T با یال‌هایی با ظرفیت بی‌نهایت متصل می‌کنیم
3. الگوریتم برش کمینه-جریان بیشینه را بین A و T اجرا می‌کنیم
4. برش کمینه شامل یال‌های (A,B) و (A,C) با مجموع هزینه 3+2=5 است

بنابراین، کمترین هزینه برای ایزوله کردن ذهن آلوده A از سایر ذهن‌ها برابر با 5 است و باید یال‌های (A,B) و (A,C) را قطع کنیم.

4.4 تحلیل الگوریتم

درستی الگوریتم

الگوریتم برش کمینه-جریان بیشینه یک الگوریتم استاندارد در نظریه گراف است که درستی آن بر اساس قضیه فورد-فالکرسون اثبات شده است. این قضیه بیان می‌کند که مقدار جریان بیشینه از s به t برابر با ظرفیت برش کمینه بین s و t است.

در مسئله ما، با اضافه کردن گره مجازی t و اتصال تمام ذهن‌های پاک به آن، مسئله ایزوله کردن ذهن آلوده از سایر ذهن‌ها به مسئله یافتن برش کمینه بین s و t تبدیل می‌شود.

پیچیدگی زمانی

1. **ساخت گراف**: $O(V+E)$
2. **الگوریتم فورد-فالکرسون**: $O(E \cdot f)$، که f مقدار جریان بیشینه است
   - با استفاده از الگوریتم ادموندز-کارپ: $O(V \cdot E^2)$
   - با استفاده از الگوریتم دینیک: $O(V^2 \cdot E)$
3. **یافتن برش کمینه**: $O(V+E)$

بنابراین، پیچیدگی زمانی کل الگوریتم با استفاده از ادموندز-کارپ $O(V \cdot E^2)$ است.

در شبکه‌های ذهنی چندجمله‌ای، اگر تعداد گره‌ها n باشد، تعداد یال‌ها می‌تواند $O(n^2)$ باشد. بنابراین، پیچیدگی زمانی در بدترین حالت $O(n^5)$ خواهد بود. اما در عمل، شبکه‌های ذهنی معمولاً تُنُک هستند (تعداد یال‌ها بسیار کمتر از n^2 است)، بنابراین الگوریتم در زمان قابل قبولی اجرا می‌شود.

بهینه‌سازی‌ها

برای بهبود کارایی الگوریتم، می‌توان از نسخه‌های بهبودیافته الگوریتم جریان بیشینه استفاده کرد:

1. **الگوریتم دینیک**: با پیچیدگی زمانی $O(V^2 \cdot E)$
2. **الگوریتم Push-Relabel**: با پیچیدگی زمانی $O(V^2 \cdot E^{(1/2)})$ یا $O(V^3)$
3. **الگوریتم MPM (Malhotra, Pramodh-Kumar, Maheshwari)**: با پیچیدگی زمانی $O(V^3)$

سوال5 – دورهمی پژوهشگران

سوال ۵: دورهمی پژوهشگران

چندین گروه پژوهشی تصمیم گرفته‌اند پس از پایان یک کنفرانس علمی، برای افزایش تعاملات اجتماعی، در یک ضیافت شام مشترک شرکت کنند. برگزارکنندگان می‌خواهند افراد به‌گونه‌ای در میزها بنشینند که هیچ دو نفر از اعضای یک گروه پژوهشی در یک میز قرار نگیرند. فرض کنید که در این ضیافت، p گروه پژوهشی شرکت کرده‌اند و گروه i شامل a_i نفر است. همچنین فرض کنید که q میز در سالن وجود دارد و ظرفیت میز j برابر b_j نفر است. یک الگوریتم طراحی کنید که بتواند بررسی کند آیا چنین چینشی ممکن است یا نه، و اگر ممکن است، نحوه‌ی نشاندن افراد را تعیین کند. در غیر این صورت، اعلام کنید که این کار امکان‌پذیر نیست.

5.1 پاسخ سوال:

تحلیل مسئله

در این مسئله:

- p گروه پژوهشی در یک ضیافت شام شرکت می‌کنند
- گروه i شامل a_i نفر است
- q میز در سالن وجود دارد و هر میز ظرفیت b_j نفر دارد
- شرط: هیچ دو نفر از اعضای یک گروه پژوهشی نباید در یک میز قرار بگیرند
- هدف: طراحی الگوریتمی که بررسی کند آیا چنین چیدمانی امکان‌پذیر است یا خیر

این مسئله را می‌توان به عنوان یک مسئله جریان شبکه مدل‌سازی کرد. به طور خاص، این یک مسئله تخصیص با محدودیت است که می‌توان آن را با استفاده از الگوریتم حداکثر جریان حل کرد.

مدل‌سازی مسئله با جریان شبکه

برای مدل‌سازی این مسئله، یک گراف جریان شبکه به صورت زیر می‌سازیم:

1. یک گره منبع s
2. برای هر گروه پژوهشی i، یک گره g_i
3. برای هر میز j، یک گره t_j
4. یک گره مقصد t
5. یال‌های زیر را با ظرفیت‌های مشخص اضافه می‌کنیم:
   - از s به هر گروه g_i، یک یال با ظرفیت a_i (تعداد اعضای گروه)
   - از هر گروه g_i به هر میز t_j، یک یال با ظرفیت 1 (حداکثر یک عضو از هر گروه در هر میز)
   - از هر میز t_j به t، یک یال با ظرفیت b_j (ظرفیت میز)

5.2 الگوریتم پیشنهادی

الگوریتم چیدمان پژوهشگران:

ورودی: تعداد گروه‌ها p، تعداد اعضای هر گروه $a_1, a_2, ..., a_p$،

تعداد میزها q، ظرفیت هر میز $b_1, b_2, ..., b_q$

خروجی: "امکان‌پذیر" یا "غیرممکن"

1. بررسی شرایط اولیه:
   a. اگر مجموع تعداد اعضای همه گروه‌ها بیشتر از مجموع ظرفیت میزها باشد:
      i. برگردان "غیرممکن"
   b. اگر تعداد اعضای یک گروه بیشتر از تعداد میزها باشد:
      i. برگردان "غیرممکن"

2. ساخت گراف جریان شبکه $G(V,E)$:
   a. یک گره منبع s اضافه کن
   b. برای هر گروه پژوهشی i $(1 \leq i \leq p)$، یک گره g_i اضافه کن
   c. برای هر میز j $(1 \leq j \leq q)$، یک گره t_j اضافه کن
   d. یک گره مقصد t اضافه کن
   e. برای هر گروه i، یک یال (s, g_i) با ظرفیت a_i اضافه کن
   f. برای هر گروه i و هر میز j، یک یال (g_i, t_j) با ظرفیت 1 اضافه کن
   g. برای هر میز j، یک یال (t_j, t) با ظرفیت b_j اضافه کن

3. با استفاده از الگوریتم فورد-فولکرسون، جریان بیشینه از s به t را محاسبه کن

4. اگر مقدار جریان بیشینه برابر با مجموع تعداد اعضای همه گروه‌ها باشد:
   a. برگردان "امکان‌پذیر"
   b. چیدمان را از جریان استخراج کن:
      i. برای هر گروه i و هر میز j، اگر در یال (g_i, t_j) جریان 1 وجود دارد،
         یک عضو از گروه i را به میز j تخصیص بده

5. در غیر این صورت:

a. برگردان "غیرممکن"

اثبات درستی الگوریتم

1. **بررسی شرایط اولیه:**

- اگر مجموع تعداد اعضای همه گروه‌ها بیشتر از مجموع ظرفیت میزها باشد، واضح است که چیدمان امکان‌پذیر نیست.
- اگر تعداد اعضای یک گروه بیشتر از تعداد میزها باشد، از آنجا که هیچ دو عضوی از یک گروه نمی‌توانند در یک میز باشند، چیدمان امکان‌پذیر نیست.

2. **مدل جریان شبکه:**

- یال‌های (s, g_i) با ظرفیت a_i نشان می‌دهد که گروه i دارای a_i عضو است.
- یال‌های (g_i, t_j) با ظرفیت 1 تضمین می‌کند که حداکثر یک عضو از گروه i در میز j قرار می‌گیرد.
- یال‌های (t_j, t) با ظرفیت b_j نشان می‌دهد که میز j حداکثر b_j نفر را می‌پذیرد.

3. **جریان بیشینه:**

- اگر جریان بیشینه برابر با مجموع تعداد اعضای همه گروه‌ها باشد، یعنی همه اعضا می‌توانند در میزها قرار بگیرند.
- در این حالت، جریان در یال‌های (g_i, t_j) نشان می‌دهد که کدام اعضای گروه‌ها در کدام میزها قرار می‌گیرند.
- اگر جریان بیشینه کمتر باشد، یعنی نمی‌توان همه اعضا را با رعایت محدودیت‌ها در میزها قرار داد.

5.3 تحلیل پیچیدگی زمانی

1. **بررسی شرایط اولیه:** $O(p + q)$

2. **ساخت گراف:**

- تعداد گره‌ها: $|V| = 1 + p + q + 1 = p + q + 2$
- تعداد یال‌ها: $|E| = p + p\cdot q + q = p + q + p\cdot q$

3. **الگوریتم فورد-فولکرسون:**

- با استفاده از الگوریتم ادموندز-کارپ: $O(|V|\cdot|E|^2) = O((p+q)\cdot(p+q+pq)^2)$
- در عمل، می‌توان از الگوریتم‌های کارآمدتر مانند دینیک استفاده کرد.

4. **استخراج چیدمان:** $O(p\cdot q)$

پیچیدگی زمانی کل: $O((p+q)\cdot(p+q+pq)^2)$

5.4 مثال

فرض کنید 3 گروه پژوهشی با تعداد اعضای $a_1=2, a_2=3, a_3=4$ و 4 میز با ظرفیت‌های $b_1=3, b_2=2, b_3=2, b_4=2$ داریم.

گام 1 :بررسی شرایط اولیه:

- مجموع تعداد اعضا: $9 = 4+3+2$
- مجموع ظرفیت میزها: $9 = 2+2+2+3$
- تعداد اعضای بزرگترین گروه (4) ≥ تعداد میزها (4)
- شرایط اولیه برقرار است.

گام 2 :ساخت گراف جریان شبکه:

- گره‌ها: $s, g_1, g_2, g_3, t_1, t_2, t_3, t_4, t$
- یال‌ها با ظرفیت‌ها:
  - (s, g_1) با ظرفیت 2
  - (s, g_2) با ظرفیت 3
  - (s, g_3) با ظرفیت 4
  - برای هر گروه i و هر میز j، (g_i, t_j) با ظرفیت 1
  - (t_1, t) با ظرفیت 3
  - $(t_2, t), (t_3, t), (t_4, t)$ هر کدام با ظرفیت 2

گام 3 :محاسبه جریان بیشینه:

- با اجرای الگوریتم فورد-فولکرسون، جریان بیشینه = 9

گام 4 :چون جریان بیشینه برابر با مجموع تعداد اعضا است، چیدمان امکان‌پذیر است.

یک چیدمان ممکن:

- میز 1: یک عضو از گروه 1، یک عضو از گروه 2، یک عضو از گروه 3
- میز 2: یک عضو از گروه 1، یک عضو از گروه 2
- میز 3: یک عضو از گروه 2، یک عضو از گروه 3
- میز 4: دو عضو از گروه 3

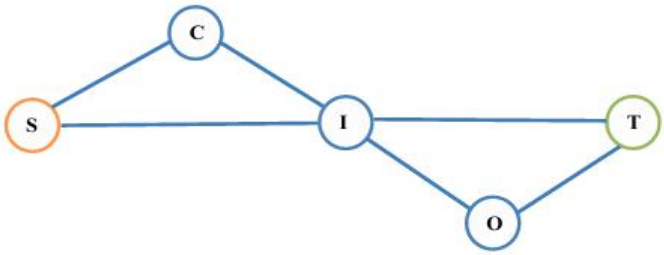
سوال6 – دورهمی پژوهشگران

در یکی از انبارهای هوشمند شرکت آمازون، کالاهای ویژه‌ای از ایستگاه‌های مختلف باید به مرکز پردازش نهایی منتقل شوند. مسیرهای ارتباطی میان ایستگاه‌ها در قالب یک شبکه‌ی جهت‌دار بدون وزن طراحی شده‌اند، به‌گونه‌ای که هر مسیر نشان‌دهنده‌ی امکان حمل بار از یک ایستگاه به ایستگاه دیگر است. برای جلوگیری از تداخل در حمل و افزایش بهره‌وری، شرکت تصمیم دارد که مسیرهای حمل برای کالاها به‌طور کامل از نظر رأس از یکدیگر مستقل باشند (یعنی هیچ دو مسیر نباید ایستگاه مشترکی داشته باشند، به‌جز ایستگاه مبدأ و مقصد).

اکنون ساختار کامل شبکه به‌صورت یک گراف جهت‌دار بدون وزن $G = (V, E)$ در اختیار شماست، به‌همراه دو رأس مشخص s و t که به‌ترتیب نشان‌دهنده‌ی ایستگاه بارگیری اولیه و مرکز نهایی پردازش هستند، و یک عدد صحیح k که $1 \le k \le n$ است. الگوریتمی طراحی کنید که در زمان چندجمله‌ای نسبت به n و k، بررسی کند آیا می‌توان k مسیر کاملاً مجزای از نظر رأس، از s به t در این گراف پیدا کرد یا نه. الگوریتم باید در صورت وجود چنین مسیرهایی، پاسخ بله و در غیر این‌صورت پاسخ خیر را بازگرداند.

برای این مسئله، می‌توانید فرض کنید که به یک الگوریتم چندجمله‌ای برای حل مسئله‌ی مسیرهای مجزای یالی دسترسی دارید. تعریف این مسئله به‌صورت زیر است: با داشتن یک گراف جهت‌دار G و دو رأس s و t ، هدف یافتن بیشینه‌ی تعداد مسیرهای مجزای یالی از s به t است. دو مسیر P_1 و P_2 از s به t در صورتی یال‌مجزا هستند که یال مشترکی نداشته باشند.

به‌عنوان مثال، در گراف زیر، دو مسیر یال مجزا از s به t وجود دارد، اما نمی توان دو مسیر کاملاً مجزای رأسی از s به t یافت.



6.1 پاسخ سوال:

تحلیل مسئله

در این مسئله، یک شبکه جهت‌دار بدون وزن G=(V,E) داریم که نشان‌دهنده مسیرهای حمل کالا بین ایستگاه‌های مختلف در شرکت آمازون است. هدف ما طراحی الگوریتمی است که بررسی کند آیا می‌توان k مسیر کاملاً مجزای رأسی (vertex-disjoint paths) از رأس s به رأس t پیدا کرد یا خیر.

مسیرهای مجزای رأسی، مسیرهایی هستند که هیچ رأس مشترکی به جز رأس مبدأ (s) و مقصد (t) نداشته باشند. این مسئله به طور مستقیم با مسئله "مسیرهای مجزای رأسی بیشینه" (Maximum Vertex-Disjoint Paths) در نظریه گراف ارتباط دارد.

6.2 الگوریتم پیشنهادی

برای حل این مسئله، می‌توانیم از قضیه مینکات-منگر (Menger's Theorem) و تبدیل آن به یک مسئله جریان شبکه استفاده کنیم. طبق این قضیه، تعداد مسیرهای مجزای رأسی بیشینه بین دو رأس s و t در یک گراف، برابر با اندازه کمینه برش رأسی (minimum vertex cut) بین s و t است.

برای حل این مسئله با استفاده از الگوریتم جریان شبکه، گراف اصلی را به یک گراف جریان تبدیل می‌کنیم:

الگوریتم یافتن مسیرهای مجزای رأسی:

ورودی: گراف جهت‌دار G=(V,E)، رأس‌های s و t، و عدد صحیح k

خروجی: "بله" اگر k مسیر مجزای رأسی از s به t وجود داشته باشد، در غیر این صورت "خیر"

1. یک گراف جریان 'G ایجاد کنید:
   a. برای هر رأس v در G (به جز s و t)، آن را به دو رأس v_in و v_out تقسیم کنید
   b. یک یال جهت‌دار (v_in, v_out) با ظرفیت 1 اضافه کنید
   c. برای هر یال (u,v) در G:
      i. اگر u=s، یک یال (s, v_in) با ظرفیت 1 در 'G اضافه کنید
      ii. اگر v=t، یک یال (u_out, t) با ظرفیت 1 در 'G اضافه کنید
      iii. در غیر این صورت، یک یال (u_out, v_in) با ظرفیت 1 در 'G اضافه کنید

2. با استفاده از الگوریتم فورد-فولکرسون، جریان بیشینه از s به t در G' را محاسبه کنید

3. اگر مقدار جریان بیشینه حداقل k باشد:

   a. از جریان محاسبه‌شده، k مسیر مجزای رأسی را استخراج کنید

   b. "بله" را برگردانید

4. در غیر این صورت، "خیر" را برگردانید

اثبات درستی الگوریتم

1. **تبدیل گراف**: با تقسیم هر رأس به دو رأس ورودی و خروجی و قرار دادن یک یال با ظرفیت 1 بین آنها، تضمین می‌کنیم که هر رأس حداکثر در یک مسیر استفاده می‌شود. این تبدیل، مسئله یافتن مسیرهای مجزای رأسی را به مسئله یافتن مسیرهای مجزای یالی تبدیل می‌کند.

2. **جریان بیشینه**: الگوریتم فورد-فولکرسون، جریان بیشینه از s به t را محاسبه می‌کند. از آنجا که ظرفیت همه یال‌ها 1 است، هر واحد جریان نشان‌دهنده یک مسیر از s به t است.

3. **استخراج مسیرها**: از جریان محاسبه‌شده، می‌توان k مسیر مجزای رأسی را استخراج کرد. این مسیرها در گراف اصلی G نیز مجزای رأسی خواهند بود.

4. **قضیه مینکات-منگر**: طبق این قضیه، تعداد مسیرهای مجزای رأسی بیشینه بین s و t برابر با اندازه کمینه برش رأسی بین s و t است. با تبدیل گراف، این مقدار برابر با جریان بیشینه در G′ خواهد بود.

6.3 تحلیل پیچیدگی زمانی

1. **ساخت گراف G′**: $O(V+E)$

2. **الگوریتم فورد-فولکرسون**: $O(E \cdot f)$، که f مقدار جریان بیشینه است

   - با استفاده از الگوریتم ادموندز-کارپ: $O(V \cdot E^2)$
   - با استفاده از الگوریتم دینیک: $O(V^2 \cdot E)$

3. **استخراج مسیرها**: $O(V \cdot E)$

بنابراین، پیچیدگی زمانی کل الگوریتم با استفاده از ادموندز-کارپ $O(V \cdot E^2)$ است.

6.4 مثال

برای درک بهتر الگوریتم، مثال ارائه شده در سؤال را بررسی می‌کنیم:

گراف G شامل رئوس S، C، I، O، T است و یال‌های آن عبارتند از: (S,C), (S,I), (C,I), (I,T), (I,O), (O,T)

فرض کنیم k=2 است، یعنی می‌خواهیم بررسی کنیم آیا 2 مسیر مجزای رأسی از S به T وجود دارد یا خیر.

مرحله 1 :ساخت گراف G'

- رئوس: S, C_in, C_out, I_in, I_out, O_in, O_out, T
- یال‌های G' با ظرفیت‌ها:
  - (C_in, C_out), (I_in, I_out), (O_in, O_out) همگی با ظرفیت 1
  - (S, C_in), (S, I_in) همگی با ظرفیت 1
  - (C_out, I_in), (I_out, T), (I_out, O_in), (O_out, T) همگی با ظرفیت 1

مرحله 2 :محاسبه جریان بیشینه

- مسیر افزایشی 1: S → I_in → I_out → T (جریان +1)
- مسیر افزایشی 2: S → C_in → C_out → I_in → I_out → O_in → O_out → T (جریان +1)

جریان بیشینه = 2

اما این دو مسیر رأس مشترک I دارند، پس مجزای رأسی نیستند! اینجا مشکلی وجود دارد.

اصلاح :در مسیر افزایشی 2، پس از استفاده از I_in → I_out در مسیر افزایشی 1، این یال دیگر ظرفیت باقیمانده ندارد و نمی‌تواند در مسیر افزایشی 2 استفاده شود.

مسیرهای افزایشی صحیح:

- مسیر افزایشی 1: S → I_in → I_out → T (جریان +1)
- مسیر افزایشی 2: S → C_in → C_out → I_in (غیرممکن، زیرا I_in → I_out ظرفیت ندارد)

بنابراین، جریان بیشینه = 1 < k=2، پس نمی‌توان 2 مسیر مجزای رأسی پیدا کرد و پاسخ "خیر" است.

این نتیجه با مثال سؤال مطابقت دارد که بیان می‌کند در گراف مثال، نمی‌توان دو مسیر کاملاً مجزای رأسی از S به T یافت.

بهینه‌سازی‌های ممکن

1. **الگوریتم‌های جریان بیشینه کارآمدتر:**

- استفاده از الگوریتم دینیک با پیچیدگی $O(V^2 \cdot E)$
- استفاده از الگوریتم پوش-ریلبل با پیچیدگی $O(V^3)$

2. **پیش‌پردازش:**

- بررسی درجه خروجی s و درجه ورودی t: اگر $k > \min(\text{outdegree}(s), \text{indegree}(t))$، مستقیماً "خیر" را برگردانید.

3. **استفاده از الگوریتم‌های تخصصی:**

- برای مسائل مسیرهای مجزای رأسی، می‌توان از الگوریتم‌های تخصصی مانند الگوریتم Even-Tarjan استفاده کرد.

نتیجه‌گیری

الگوریتم پیشنهادی با استفاده از تکنیک تبدیل گراف و الگوریتم جریان بیشینه، یک راه‌حل کارآمد برای مسئله یافتن مسیرهای مجزای رأسی ارائه می‌دهد. این الگوریتم تضمین می‌کند که اگر k مسیر مجزای رأسی وجود داشته باشد، آنها را پیدا می‌کند، و در غیر این صورت، اعلام می‌کند که چنین مسیرهایی وجود ندارند.

با توجه به کاربرد عملی این مسئله در شبکه حمل و نقل شرکت آمازون، این الگوریتم می‌تواند به طراحی مسیرهای مستقل برای حمل کالا کمک کند و از تداخل در حمل و افزایش بهره‌وری جلوگیری نماید.

((پایان))